

My to do app must contain at least three tests that exercise real behaviour, runnable by a single documented command. Help me plan how i would write these tests

Since your app is a **Next.js + React + Prisma + SQLite** todo application, the best approach is to write tests that verify **user behaviour**, not implementation details. Your assignment specifically says the tests should **exercise real behaviour**, which usually means interacting with the UI or API the way a user would.

Step 1: Choose a testing framework

A common setup is:

- **Vitest** – test runner
- **React Testing Library** – render components and simulate user interactions
- **@testing-library/user-event** – realistic clicks and typing

If your assignment requires full end-to-end testing, you could use Playwright instead, but for most university projects, Vitest + React Testing Library is sufficient.

You should be able to run everything with a single command such as:

Bash

```
npm test
```

or

Bash

```
npm run test
```

Step 2: Decide what behaviour to test

Think about the most important user actions in your application.

Test 1 – Creating a Todo

Behaviour

A user enters a title and submits the form.

What to verify

- The new todo appears in the list.
- The input is cleared afterwards.

Example scenario:

Render the page

Type:

"Finish assignment"

Click Add

Expect:

✓ "Finish assignment" is visible

✓ Input textbox is empty

This tests that:

- typing works
- clicking works
- state updates
- rendering updates

Test 2 – Archive Filter

You were just debugging this feature, so it's a perfect candidate.

Behaviour

The user switches between Active, Archived, and All.

Example:

Initial todos

Task A (active)

Task B (archived)

Tests:

Click "Archived"

Expect:

Task B visible

Task A hidden

Then

Click "Active"

Expect:

Task A visible

Task B hidden

Then

Click "All"

Expect:

Both tasks visible

This verifies your filtering logic actually updates the UI.

Test 3 – Search

Example data

Buy milk

Study React

Wash car

User types

React

Expected

Study React ✓

Buy milk X

Wash car X

This confirms your search filter behaves correctly.

Alternative tests

If you already have these features, you could instead test:

Completing a todo

Click checkbox

Expect:

✓ todo marked completed

Deleting a todo

Click Delete

Expect:

Todo removed from screen

Editing a todo

Click Edit

Change title

Save

Expect:

Updated title visible

API test

If your app has API routes:

POST /api/todos

Expect:

201 Created

GET /api/todos

Expect:

new todo returned

Test structure

A typical React Testing Library test looks like this:

```
describe("Todo App", () => {  
  
  test("adds a new todo", () => {  
    Render app  
  
    Type into textbox  
  
    Click Add button  
  
    Expect todo appears  
  });  
  
});
```

Notice that the test reads almost like English.

What your lecturer is likely looking for

These tests should demonstrate that your application works from a user's perspective. A good set would be:

Test	Behaviour	Why it's valuable
Create Todo	User adds a task	Verifies the core feature of the app
Archive Filter	User changes archive filter	Verifies filtering logic and UI updates
Search	User searches for a task	Verifies search functionality and rendering

Together, these cover **creating data**, **filtering data**, and **finding data**—three key behaviours in your todo application.

Because your app already includes archive filtering, search, and CRUD operations, these three tests fit naturally with your existing functionality and require minimal extra code.



Ok so ive implemented the tests but now i have this error even though the file exists: RERUN __tests__/Home.test.tsx x1

➤ __tests__/Home.test.tsx (0 test)

Failed Suites 1

Show more ▼

The important part of the error is this:

```
Error: Failed to resolve import "@app/page"
```

This is **not saying** `app/page.tsx` **doesn't exist**. It's saying **Vitest/Vite doesn't know what `@` means**.

Step 1. Verify the file actually exists

First confirm your project looks something like:

```
SDP-Lab1/
|
├─ app/
|   └─ page.tsx
|
├─ __tests__/
|   └─ Home.test.tsx
|
├─ tsconfig.json
├─ vite.config.ts
└─ package.json
```

If `page.tsx` is somewhere else (for example `src/app/page.tsx`), the import needs to match that.

Step 2. Check whether `@` works in your application

Look inside one of your components.

Do you already have imports like

TypeScript

```
import prisma from "@lib/prisma";
```

or

TypeScript

```
import Button from "@components/Button";
```

If those work when running Next.js but **not in Vitest**, then your alias is only configured for Next.js.

Step 3. Check `tsconfig.json`

You should have something similar to

JSON

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["./*"]
    }
  }
}
```

or, if your code lives under `src` :

JSON

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": [".src/*"]
    }
  }
}
```

Step 4. Configure Vitest/Vite

Even if TypeScript understands the alias, **Vitest won't unless you tell Vite about it.**

Open `vite.config.ts` .

It should contain something like:

TypeScript

```
import { defineConfig } from "vitest/config";
import path from "path";

export default defineConfig({
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "."),
    },
  },
  test: {
    environment: "jsdom",
  },
});
```

If your project uses `src` , then use

TypeScript

```
alias: {  
  "@": path.resolve(__dirname, "src"),  
}
```

Step 5. Quick test

Instead of

TypeScript

```
import Home from "@app/page";
```

temporarily try

TypeScript

```
import Home from "../app/page";
```

If that works immediately, you've confirmed the problem is **only the alias configuration**.

The most likely cause

Since you're using **Next.js 16** with **Vitest**, I'd say there's about a **90% chance** the issue is that **@ is configured in tsconfig.json but not in vite.config.ts**, so Next.js understands the alias while Vitest does not.

Can you paste these three files?

1. vite.config.ts
2. tsconfig.json
3. The first 10 lines of __tests__/Home.test.tsx

With those, I can pinpoint the exact configuration issue.



ChatGPT can make mistakes. Check important info.